Doc. no.:	P2174R0
Date:	2020-5-15
Audience:	EWG Incubator
Reply-to:	Zhihao Yuan <zy at miator dot net>

Compound Literals

This paper proposes to standardize an existing practice, namely compound literals, available in GCC and Clang for C++ language modes. It shares the syntax with C99 compound literals but produces prvalue.

Motivation

When teaching *new-expression*s, the author found that it is useful to let a student correlate creating an object of automatic storage and an object of dynamic storage:

```
struct Point
{
    int x, y;
};

new Point(3, 4)  // returns a pointer to a Point object on heap, because
    Point(3, 4)  // is an anonymous Point object on stack, while
  Point a(3, 4); // can give a name to such an object
```

But this encounters some trouble when applying to dynamic arrays. In hypothesis, the author wants to see

```
new double[]{.3, .2, .1}  // returns a pointer to a dynamic array, because
    double[]{.3, .2, .1}  // is an anonymous array on stack, while
  double a[]{.3, .2, .1}; // can give a name to such an array
```

There is no such grammar for the second line, but it is entirely acceptable to spell such an expression as follows.

```
(double[]){.3, .2, .1}
```

Because:

1. It is intuitive to group tokens with parenthesis.
2. It works already in GCC and Clang.
3. C has the same syntax.

Some people, including experts, believe that it is a part of C++ because all relevant C++ compilers, except MSVC, support the proposed syntax. We occasionally find open-source contributors reverting uses of compound literals in people's C++ code.

However, the compilers powered by the EDG frontend seem to produce lvalues for the proposed syntax. In the following code snippet,

```
auto&& x = (double[]){ 3, 4, 5 };
```

`x` is of type `double(&&)[3]` in GCC and Clang, but is of type `double(&)[3]` in NVCC and Intel C++ Compiler. Hence, there is a widely spread existing practice, but with a slightly diverging behavior, which can be better-off if standardized.

Discussion

There is a workaround for it…

```
using arr_t = double[];
auto&& x = arr_t{ 3, 4, 5 };
```

It's silly.

Why don't you define an alias template?

```
template<class T> using id = T;
auto&& x = id<double[]>{ 3, 4, 5 };
```

It's not only silly but also expert only. The author has never seen the code outside [class.temporary]/6.8 (http://eel.is/c++draft/class.temporary#6.example-1).

Let's focus more on the portability aspect of our situation.

Why producing prvalue rather than closely matching C99?

Given the fact that a typedef followed by *braced-init-list* produces prvalue in C++,

```
using arr_t = double[];
auto&& x = arr_t{ 3, 4, 5 };  // double(&&)[3]
```

it would be a wat (https://www.destroyallsoftware.com/talks/wat) for a parentasised typedef followed by *braced-init-list* to produce lvalue:

```
using arr_t = double[];
auto&& x = (arr_t){ 3, 4, 5 }; // double(&)[3] ??
```

To quote Richard,

> If we allow this syntax, I think the only consistent and rational semantic model is for `T{inits}` to mean the same thing as `(T){inits}`, and we should be cognizant that we are somewhat diverging from C's semantics in adopting that model.

The "Initializer lists"[1] paper discussed an additional issue in ch. 5.5.

Wording

The wording is relative to N4861.

Extend the grammar:

> *cast-expression*:
> *unary-expression*
> `(` *type-id* `)` *cast-expression*
> `(` *type-id* `)` *braced-init-list*

Insert a new paragraph after [expr.cast]/1 (http://eel.is/c++draft/expr.cast#1):

> The result of the expression `(T)` *cast-expression* is of type `T`. The result is an lvalue if `T` is an lvalue reference type or an rvalue reference to function type and an xvalue if `T` is an rvalue reference to object type; otherwise the result is a prvalue. *[Note*: […] — *end note]*
> If an expression is of form `(` *type-id* `)` *braced-init-list*, let init be the *braced-init-list* and `T` be the *type-id*. The expression is equivalent to `T` init ([expr.type.conv] (http://eel.is/c++draft/expr.type.conv)).

References

1. Stroustrup, Bjarne and Gabriel Dos Reis. N2215 *Initializer lists (Rev. 3)*. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2215.pdf (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2215.pdf) ↩